

Java Programming Immersion 2024 Lecture 2

Faculty Name :

Dr. Vinod Jain, Mr. Ankit Prakash

Arrays in Java

- An array is a group of like-typed variables that are referred to by a common name.
- Elements of an array are stored at contiguous memory locations.
- Arrays can be easily processed by using loops.

Arrays in Java

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

Array Length = 9

First Index = 0

Last Index = 8

Arrays in Java

- Arrays in Java work differently than they do in C/C++.
- Following are some important point about Java arrays.
 1. In Java all arrays are dynamically allocated.
 2. Since arrays are objects in Java, we can find their length using member length.
 3. A Java array variable can also be declared like other variables with [] after the data type.
 4. The variables in the array are ordered and each have an index beginning from 0.
 5. The **size** of an array must be specified by an integer value.

Arrays in Java : Declaring arrays

```
int intArray[]; //declaring array
```

```
intArray = new int[20]; // allocating memory to array
```

OR

```
int[] intArray = new int[20]; // combining both statements in one
```

Name of array
(note that all
elements of this
array have the
same name, c)

c[0]

-45

c[1]

6

c[2]

0

c[3]

72

c[4]

1543

c[5]

-89

c[6]

0

c[7]

62

c[8]

-3

c[9]

1

c[10]

6453

c[11]

78

Index (or subscript)
of the element in
array c

Fig. 7.1 A 12-element array.

Multidimensional Arrays

- **These arrays are used to define tabular data**
- Multidimensional arrays are **arrays of arrays** with each element of the array holding the reference of other array.
- `int[][] matrix = new int[10][20];` //a 2D array or matrix
- `int[][][] matrix3d = new int[10][20][10];` //a 3D array

Multidimensional Arrays

- In Java, *multidimensional arrays* are actually arrays of arrays.
- `int twoD[][] = new int[4][5];`
- It will declare an array to 4 rows and 5 columns.

7.9 Multidimensional Arrays (Cont.)

- Creating multidimensional arrays

- Can be allocated dynamically

- 3-by-4 array

```
int b[][];  
b = new int[ 3 ][ 4 ];
```

- Rows can have different number of columns

```
int b[][];  
b = new int[ 2 ][ ]; // allocate rows  
b[ 0 ] = new int[ 5 ]; // allocate row 0  
b[ 1 ] = new int[ 3 ]; // allocate row 1
```

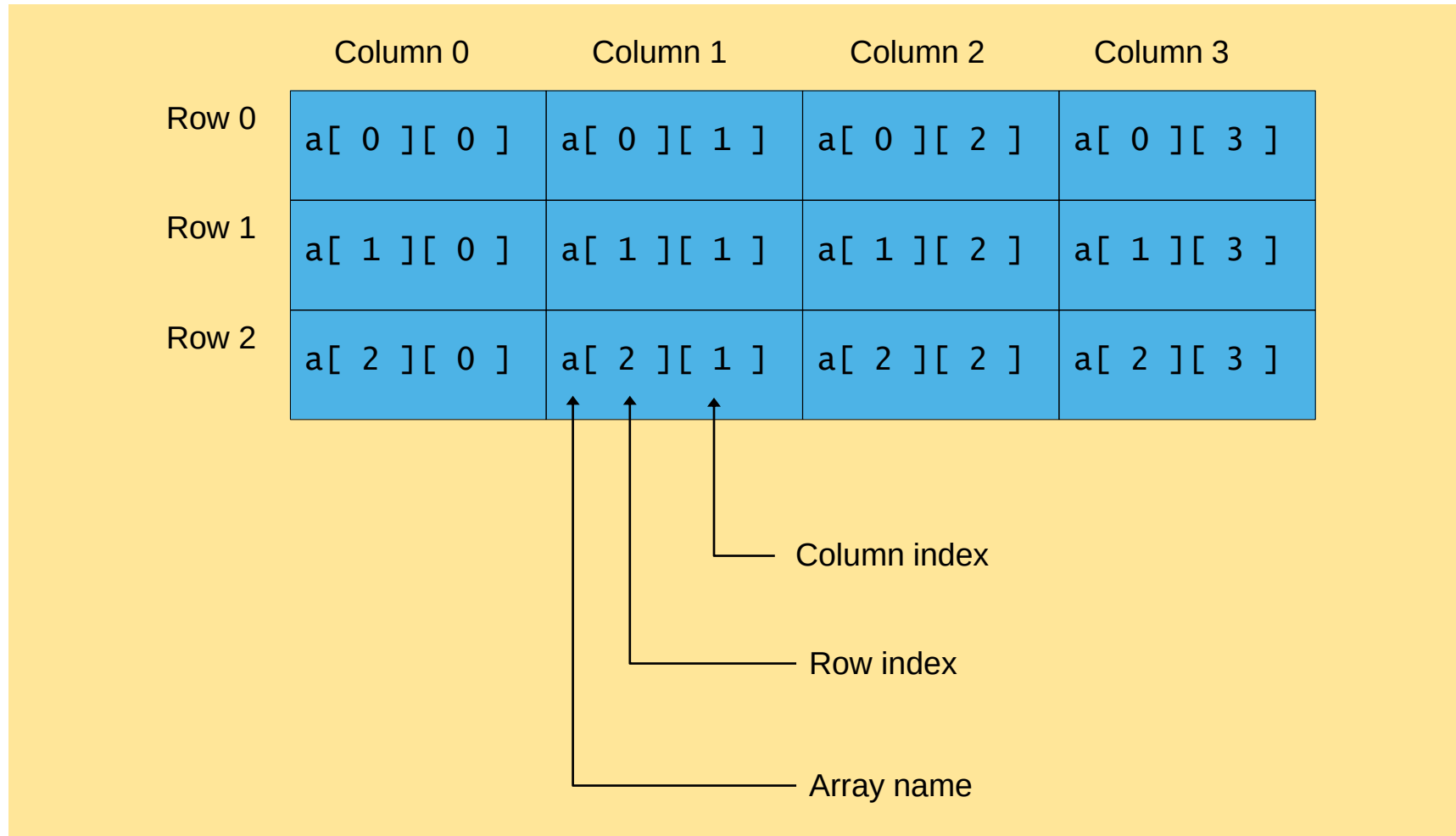


Fig. 7.13 Two-dimensional array with three rows and four columns.

Command Line Arguments

- A command-line argument is an information that directly follows the program's name on the command line when it is executed.
- They are stored as strings in the String array passed to main().
- We can use the arguments passed at command line using array of strings passed as argument to main function.

Module-1-Section-3-Java-v1.pptx - PowerPoint

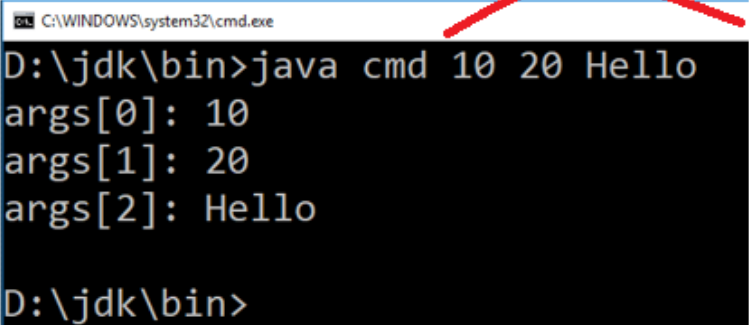
File Home Insert Design Transitions Animations Slide Show Review View Tell me what you want to do... Sign in Share

Clipboard Slides Font Paragraph Drawing Editing

WAP to print all command line arguments entered at command line

Command Line Arguments

```
public class cmd {  
    public static void main(String args[]) {  
        for(int i = 0; i<args.length; i++) {  
            System.out.println("args[" + i + "]: " + args[i]);  
        }  
    }  
}
```



Slide 12 of 13

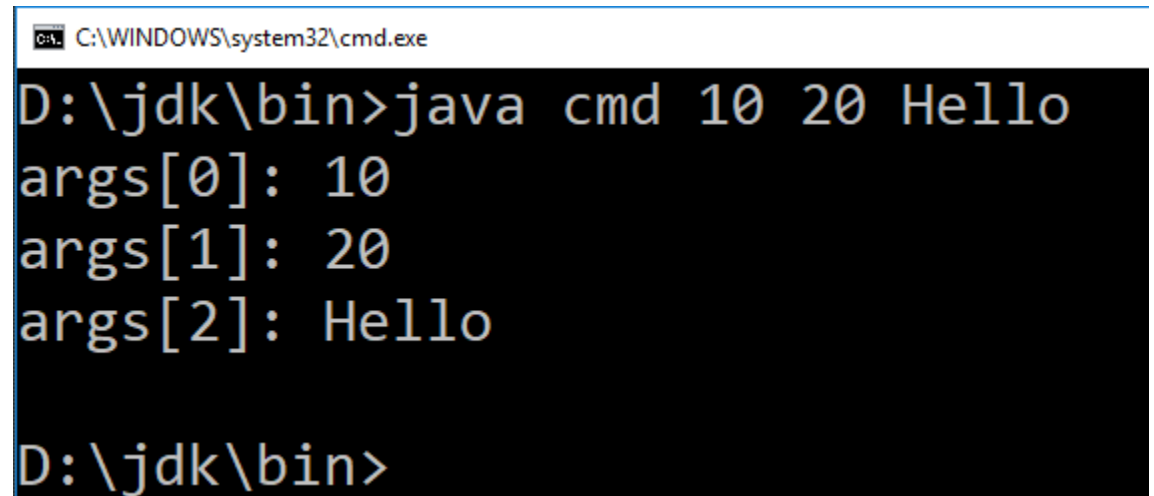
Notes Comments

73%

Activate Windows
Go to Settings to activate Windows.

WAP to print all command line arguments entered at command line

```
public class cmd {  
    public static void main(String args[]) {  
        for(int i = 0; i<args.length; i++) {  
            System.out.println("args[" + i + "]: " + args[i]);  
        }  
    }  
}
```

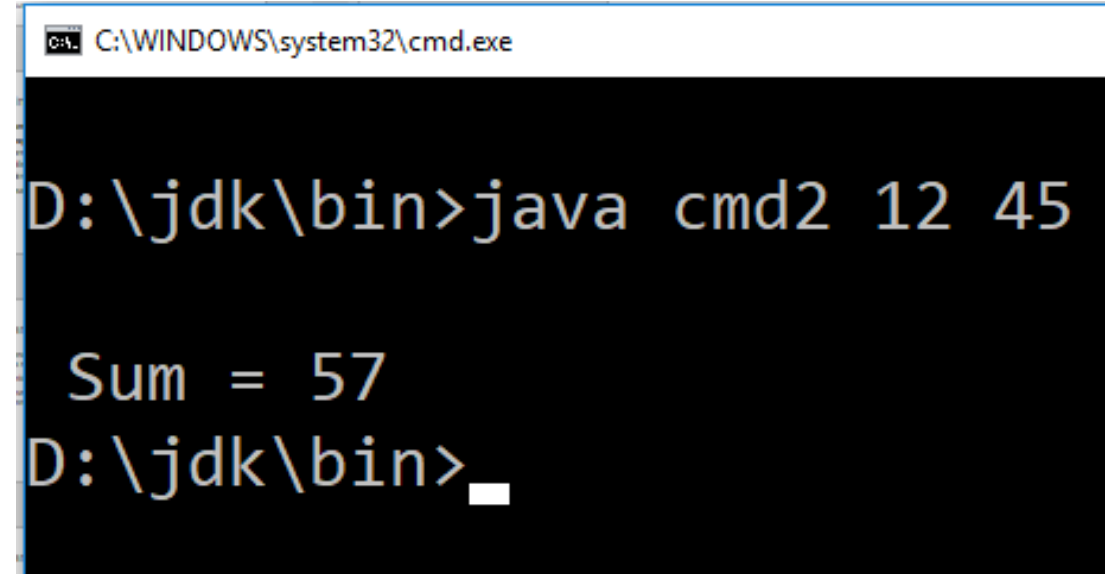


The screenshot shows a Windows command prompt window with the title bar "C:\WINDOWS\system32\cmd.exe". The command prompt shows the following sequence of text: a prompt "D:\jdk\bin>" followed by the command "java cmd 10 20 Hello", and then the output of the program: "args[0]: 10", "args[1]: 20", and "args[2]: Hello". A final prompt "D:\jdk\bin>" is shown at the bottom.

```
C:\WINDOWS\system32\cmd.exe  
D:\jdk\bin>java cmd 10 20 Hello  
args[0]: 10  
args[1]: 20  
args[2]: Hello  
D:\jdk\bin>
```

// Sum of two numbers passed from command line

```
class cmd2 {  
    public static void main(String args[]) {  
        int x,y,sum;  
        x = Integer.parseInt(args[0]);  
        y = Integer.parseInt(args[1]);  
  
        sum = x+y;  
        System.out.print("\n Sum = " + sum);  
    }  
}
```

A screenshot of a Windows command prompt window. The title bar at the top reads "C:\WINDOWS\system32\cmd.exe". The command prompt shows the directory "D:\jdk\bin" and the command "java cmd2 12 45" being executed. The output of the program is "Sum = 57". The prompt then shows "D:\jdk\bin>" with a cursor, indicating the program has finished execution.

```
C:\WINDOWS\system32\cmd.exe  
D:\jdk\bin>java cmd2 12 45  
Sum = 57  
D:\jdk\bin>
```

String

A *string* is a sequence of characters.

Java implements strings as objects of type **String**.

Eg. `String str = new String();`

Here `str` is an object of type class `String`.

Many constructors and functions are defined in string class that can be used to perform operations strings.

String

The String Constructors

`String s = new String(); // Create Empty String`

`String s2 = new String("Hello String 2"); // Create and initialize string`

OR

`String s3 = "Hello String 3";`

String

The String Constructors

// Initialize a string with array of characters

```
String(char chars[ ])
```

//Eg.

```
char chars[] = { 'a', 'b', 'c' };
```

```
String s4 = new String(chars);
```

String

The String Constructors

String(String strObj) // Initialize string with another string object
// Eg.

```
String s5 = new String("Hello String");
```

```
String s6 = new String(s5);
```

// And Many more constructors

String Functions

- Various functions are available in String class which are used to manipulate strings easily in JAVA
- These functions are used to
 1. Getting string length
 2. String concatenation
 3. String conversions
 4. Character Extraction
 5. String comparison
 6. Searching
 7. and many more

String Functions

- length()
- toString()
- charAt()
- **getChars()**
- equals() and equalsIgnoreCase()
- **compareTo()**
- indexOf()
- lastIndexOf()
- substring()
- valueOf()
- toLowerCase()
- toUpperCase()
- And many more

length()

The length of a string is the number of characters that it contains.

To obtain this value, call the **length()** method, shown here:

```
public class string2 {  
    public static void main(String args[]) {  
        String s = new String("Hello String");  
        System.out.println(s.length());  
    }  
}
```

length()

The length of a string is the number of characters that it contains.

To obtain this value, call the **length()** method, shown here:

```
public class string2 {  
    public static void main(String args[]) {  
        String s = new String("Hello String");  
        System.out.println(s.length());  
    }  
}
```

run:

12

BUILD SUCCESSFUL (total time: 0 seconds)

toString()

it is the means by which you can determine the string representation for objects of classes that you create.

```
public class string2 {  
    public static void main(String args[]) {  
        Integer xobj = new Integer("100");  
        String s = xobj.toString();  
        System.out.println(s);  
    }  
}
```

toString()

it is the means by which you can determine the string representation for objects of classes that you create.

```
public class string2 {  
    public static void main(String args[]) {  
        Integer xobj = new Integer("100");  
        String s = xobj.toString();  
        System.out.println(s);  
    }  
}
```

run:

100

BUILD SUCCESSFUL (total time: 0 seconds)

charAt()

- This function is used to extract a character at a given index from the string.
- It has this general form:
- `char charAt(int where)`

```
String str = new String("Hello String ");  
char ch;  
ch = str.charAt(6);  
System.out.println("\n ch = " + ch);
```

charAt()

- This function is used to extract a character at a given index from the string.
- It has this general form:
- `char charAt(int where)`

```
String str = new String("Hello String ");  
char ch;  
ch = str.charAt(6);  
System.out.println("\n ch = " + ch);
```

run:

ch = S

BUILD SUCCESSFUL (total time: 0 seconds)

getChars()

- If you need to extract more than one character at a time, you can use the **getChars()** method.
- It has this general form:
- `void getChars(int sourceStart, int sourceEnd, char target[ ], int targetStart)`

```
String s = "This is a demo of the getChars method.";
```

```
int start = 10;
```

```
int end = 14;
```

```
char buf[] = new char[end - start];
```

```
s.getChars(start, end, buf, 0);
```

```
System.out.println(buf);
```

getChars()

- If you need to extract more than one character at a time, you can use the **getChars()** method.
- It has this general form:
- `void getChars(int sourceStart, int sourceEnd, char target[ ], int targetStart)`

```
String s = "This is a demo of the getChars method.";
```

```
int start = 10, end = 14;
```

```
char buf[] = new char[end - start];
```

```
s.getChars(start, end, buf, 0);
```

```
System.out.println(buf);
```

run:

demo

BUILD SUCCESSFUL (total time: 0 seconds)

`equals( )` and `equalsIgnoreCase( )`

- To compare two strings for equality, use **`equals( )`**. It returns true if the calling string is equal to the string passed as argument.
- It has this general form:
- `boolean equals(Object str)`

- To perform a comparison that ignores case differences, call `equalsIgnoreCase( )`.
- It has this general form:
- `boolean equalsIgnoreCase(String str)`

`equals( )` and `equalsIgnoreCase( )`

```
String s1 = "Hello";
```

```
String s2 = "Hello";
```

```
String s3 = "Good-bye";
```

```
String s4 = "HELLO";
```

```
System.out.println(s1 + " equals " + s2 + " -> " + s1.equals(s2));
```

```
System.out.println(s1 + " equals " + s3 + " -> " + s1.equals(s3));
```

```
System.out.println(s1 + " equals " + s4 + " -> " + s1.equals(s4));
```

```
System.out.println(s1 + " equalsIgnoreCase " + s4 + " -> " +  
s1.equalsIgnoreCase(s4));
```

equals() and equalsIgnoreCase()

```
String s1 = "Hello";    String s2 = "Hello";  
String s3 = "Good-bye";    String s4 = "HELLO";  
System.out.println(s1 + " equals " + s2 + " -> " + s1.equals(s2));  
System.out.println(s1 + " equals " + s3 + " -> " + s1.equals(s3));  
System.out.println(s1 + " equals " + s4 + " -> " + s1.equals(s4));  
System.out.println(s1 + " equalsIgnoreCase " + s4 + " -> " +  
s1.equalsIgnoreCase(s4));
```

run:

Hello equals Hello -> true

Hello equals Good-bye -> false

Hello equals HELLO -> false

Hello equalsIgnoreCase HELLO -> true

BUILD SUCCESSFUL (total time: 0 seconds)

compareTo()

- you need to know which is *less than*, *equal to*, or *greater than* the next.
- A string is less than another if it comes before the other in dictionary order. A string is greater than another if it comes after the other in dictionary order.
- The **String** method **compareTo()** serves this purpose.
- It has this general form:
- `int compareTo(String str)`

Value	Meaning
Less than zero	The invoking string is less than <i>str</i> .
Greater than zero	The invoking string is greater than <i>str</i> .
Zero	The two strings are equal.

compareTo()

```
String s1 = "Hello";  
String s2 = "Good-bye";  
if(s1.compareTo(s2) < 0)  
{  
    System.out.print("\n " + s1 + " is less than " + s2);  
}  
else  
{  
    System.out.print("\n " + s2 + " is less than " + s1);  
}
```

compareTo()

```
String s1 = "Hello";
```

```
String s2 = "Good-bye";
```

```
if(s1.compareTo(s2) < 0)
```

```
{
```

```
    System.out.print("\n " + s1 + " is less than " + s2);
```

```
}
```

```
else
```

```
{
```

```
    System.out.print("\n " + s1 + " is less than " + s2);
```

```
}
```

run:

Hello is less than Good-byeBUILD SUCCESSFUL (total time: 0 seconds)

Searching Strings : indexOf(), lastIndexOf()

- The **String** class provides two methods that allow you to search a string for a specified character or substring:
- **indexOf()** Searches for the first occurrence of a character or substring and **lastIndexOf()** return the last occurrence. **lastIndexOf()**

```
String s = "Honesty is the best policy";
```

```
    System.out.println(s);
```

```
    System.out.println("indexOf(t) = " +  
s.indexOf('t'));
```

```
    System.out.println("lastIndexOf(t) = " +  
s.lastIndexOf('t'));
```

Searching Strings : indexOf(), lastIndexOf()

- String s = "Honesty is the best policy";
 System.out.println(s);
 System.out.println("indexOf(t) = " +
 s.indexOf('t'));
 System.out.println("lastIndexOf(t) = " +
 s.lastIndexOf('t'));

run:

Honesty is the best policy

indexOf(t) = 5

lastIndexOf(t) = 18

substring()

- You can extract a substring using **substring()**.
- **substring()** allows you to specify the beginning and ending index of the substring:
- `String substring(int startIndex, int endIndex)`
- Here, *startIndex* specifies the beginning index, and *endIndex* specifies the stopping point.
- `String org = "This is a test. This is, too.";`
- `String substr = org.substring(10, 14);`
- `System.out.println("\n Sub string = " + substr);`

substring()

- You can extract a substring using **substring()**.
- **substring()** allows you to specify the beginning and ending index of the substring:
- `String substring(int startIndex, int endIndex)`
- Here, *startIndex* specifies the beginning index, and *endIndex* specifies the stopping point.
- `String org = "This is a test. This is, too.";`
- `String substr = org.substring(10, 14);`
- `System.out.println("\n Sub string = " + substr);`
- run:
- Sub string = test
- BUILD SUCCESSFUL (total time: 0 seconds)

valueOf()

- It is used to convert any built-in data type to String object.
- It is a static method that is overloaded within **String** for all of Java's built-in types so that each type can be converted properly into a string.

```
String str;
```

```
int x=100;
```

```
str = String.valueOf(x);
```

```
System.out.print("\n str of x = " + str);
```

```
float f = 2.5F;
```

```
str = String.valueOf(f);
```

```
System.out.print("\n str of f = " + str);
```

valueOf()

- String str;
int x=100;
str = String.valueOf(x);
System.out.print("\n str of x = " + str);
float f = 2.5F;
str = String.valueOf(f);
System.out.print("\n str of f = " + str);

run:

str of x = 100

str of f = 2.5

BUILD SUCCESSFUL (total time: 0 seconds)

toLowerCase() toUpperCase()

- The method **toLowerCase()** converts all the characters in a string from uppercase to lowercase.
- The **toUpperCase()** method converts all the characters in a string from lowercase to uppercase.
- Nonalphabetical characters, such as digits, are unaffected.
- Here are the general
- forms of these methods:
- String toLowerCase()
- String toUpperCase()

toLowerCase() toUpperCase()

```
String s = "This is a test.";
System.out.println("Original: " + s);
String upper = s.toUpperCase();
String lower = s.toLowerCase();
System.out.println("Uppercase: " + upper);
System.out.println("Lowercase: " + lower);
```

- run:
- Original: This is a test.
- Uppercase: THIS IS A TEST.
- Lowercase: this is a test.
- BUILD SUCCESSFUL (total time: 0 seconds)

- `String[] words = str.split(" ");`

Java StringBuffer Class

- Java StringBuffer class is used to create mutable (modifiable) String objects.
- The StringBuffer class in Java is the same as String class except it is mutable i.e. it can be changed.

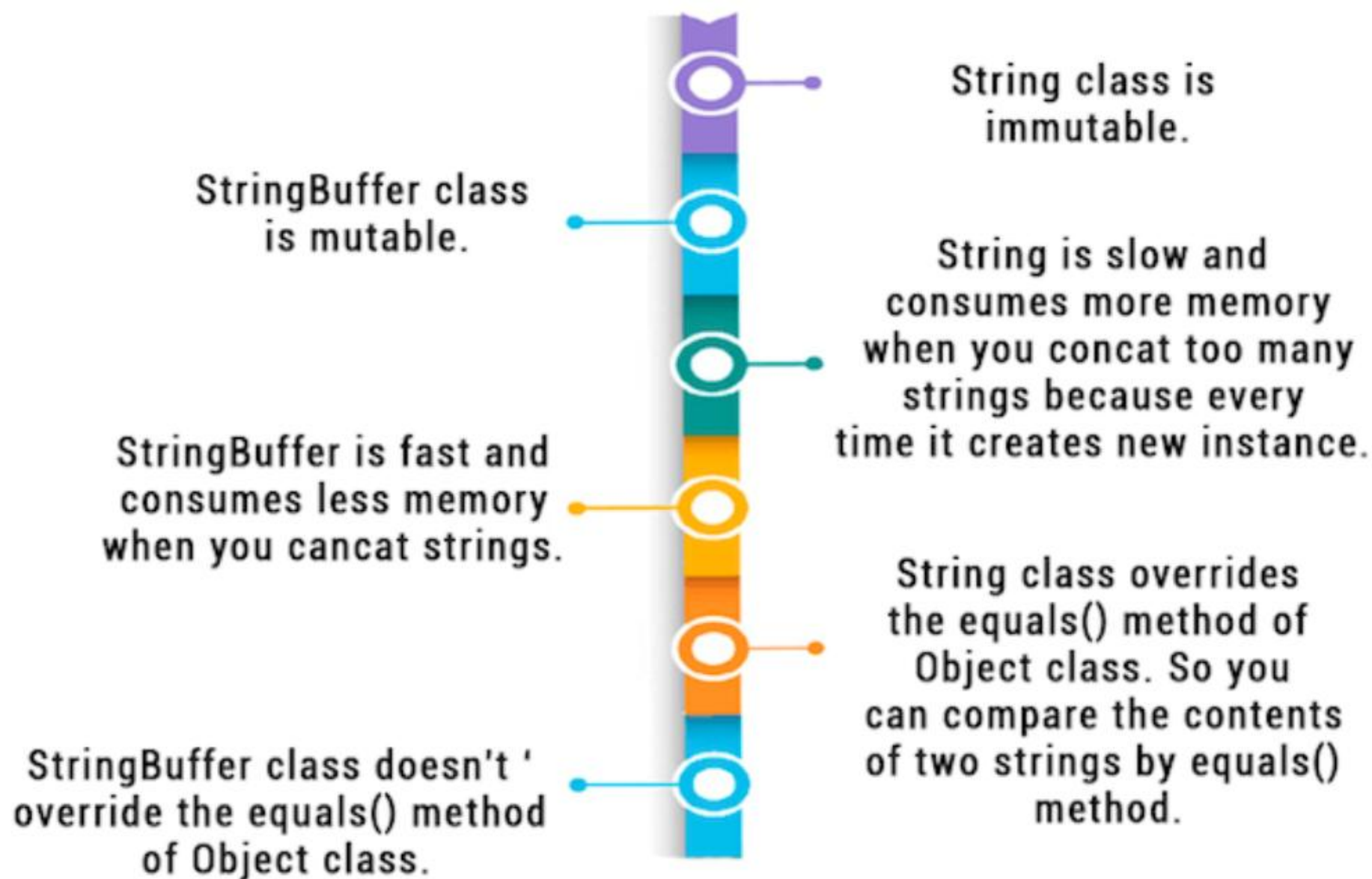
String Vs StringBuffer Class

- `String sb1 = new String("Hello ");`
 - `sb1.append("Java");` // Error - can not find symbol append
 - `System.out.println(sb1);`
-
- `StringBuffer sb = new StringBuffer("Hello ");`
 - `sb.append("Java");` // now original string is changed
 - `System.out.println(sb);`

Java StringBuffer Class

- Important methods of StringBuffer class
- `append(String s)`
- `insert(int offset, String s)`
- `replace(int startIndex, int endIndex, String str)`
- `delete(int startIndex, int endIndex)`
- `reverse()`
- `charAt(int index)`
- `length()`
- `substring(int beginIndex)`
- `substring(int beginIndex, int endIndex)`

StringBuffer vs String



Java StringBuilder Class

- Java **StringBuilder** class is used to create mutable (modifiable) String.
- The Java StringBuilder class is same as StringBuffer class except that it is non-synchronized.
- It is available since JDK 1.5.

StringBuilder Vs. StringBuffer: Differences and Applications

Feature	StringBuilder	StringBuffer
Thread-Safety	Not thread-safe	Thread-safe
Performance	More efficient	Slightly less efficient due to thread-safety
Synchronization	Not synchronized	Synchronized
Mutability	Mutable	Mutable
Usage	Suitable for single-threaded environments	Suitable for multi-threaded environments
Modifiability	Supports modification and appending	Supports modification and appending
Memory Allocation	Less memory allocation overhead	Slightly more memory allocation overhead
Recommended in	Non-concurrent scenarios, performance-critical operations	Concurrent scenarios, thread-safe environments